

INSTITUTO TECNOLÓGICO DE COSTA RICA

CAMPUS CENTRAL CARTAGO

Escuela de Ingeniería en Computación

Principios de Sistemas Operativos

Prof. Armando Arce Orozco

PROYECTO 2

Estudiantes:

Isaac Rojas Robles | 2022104884

Keren Fuentes Guevara | 2021024472

31 Abril 2026

I Semestre 2026

Índice

Introducción.....	2
Descripción del problema.....	2
Solución propuesta.....	2
Ejecución de pruebas.....	4
Análisis de resultados.....	4
Conclusiones.....	6

Introducción

En este proyecto se implementa una librería en C y un programa de línea de comando (CLI) para indexar archivos JSON y luego recuperar fragmentos del JSON a partir de una búsqueda con expresiones regulares. La motivación principal es evitar el costo de parsear un JSON grande cada vez que se necesita un subconjunto de datos.

La estrategia del proyecto toma ideas parecidas a enfoques de archivos indexados, pero aplicada a JSON, en lugar de guardar registros binarios, se conserva el archivo JSON original y se construye un índice que permite localizar valores sin reinterpretar toda la estructura.

En concreto, el índice contiene para cada elemento del JSON:

- Su ruta dentro del árbol con la raíz.
- El rango de bytes donde se ubica el valor en el archivo original.
- El tipo del valor (objeto, arreglo, string, número, bool o null).

Con ese índice, una búsqueda se reduce a:

1. Filtrar las rutas almacenadas aplicando una expresión regular.
2. Extraer directamente del JSON original los bytes.
3. Devolver los resultados en formato JSON.

Como parte de las restricciones típicas del proyecto, la solución evita librerías externas para el parseo: se usa un parser que asume JSON bien formado y cubre casos generales.

Este enfoque tiene como beneficios que se reduce el tiempo de consulta y se separan las responsabilidades de manera eficiente.

Descripción del problema

La descripción del problema se puede encontrar en este [enlace](#).

Solución propuesta

La solución se divide en dos operaciones principales:

1. Construcción del índice

- Se abre el JSON en modo binario.
- Se recorre el documento con un parser que va emitiendo eventos de elementos encontrados.

Para cada elemento se construye:

- path: ruta absoluta del elemento (con raíz \$).
- offset_start: byte donde empieza el valor.

- `offset_end`: byte siguiente al final del valor.
- `type`: tipo del valor (objeto, arreglo, string, número, bool, null).

2. Búsqueda y extracción

- Se carga el índice `.jnx` completo en memoria.
- Se compila el patrón con regex POSIX y se aplica contra cada path.
- Para cada coincidencia se extrae el fragmento del `.json` original con los offsets almacenados.
- Se imprime un arreglo JSON.
- Si no hay coincidencias, la salida es exactamente `[]`.

En términos de arquitectura, el procesamiento es un pipeline, este diseño permite probar cada parte por separado y además validar el comportamiento completo del sistema con pruebas de integración.

Formato de paths

Los paths se representan con raíz `$` y un estilo cercano a `JSONPath`, donde se administra el acceso al campo de objeto y a los elementos del arreglo. Este formato se usa para el filtrado por expresión regular, en la ejecución, el uso de regex sobre paths permite expresar búsquedas amplias sin necesidad de conocer exactamente todos los índices. Aunque al tratarse de expresiones regulares, el poder de búsqueda depende de cómo se definan los patrones.

Formato del índice `.jnx`

El archivo `.jnx` es un formato binario simple:

Header fijo (`jnx_header_t`):

- `entry_count`: cantidad total de entradas
- `source_size`: tamaño del `.json` al momento de indexar

Entradas (`jnx_entry_t`) en tamaño fijo:

- `path[512]`
- `offset_start / offset_end`
- `type`

Ejecución de pruebas

Las pruebas existentes se dividen en pruebas unitarias, que validan el comportamiento de partes concretas del proyecto como el parser y demás, y pruebas de integración, que validan la integridad del binario.

Esto es relevante porque el Makefile y el script de integración están escritos para un entorno tipo Linux/Unix, aunque también pueden ser ejecutados desde Windows con la herramienta MSYS2. Los comandos para ejecutar las pruebas son *make clean* y *make test*.

Los aspectos que se validaron en las pruebas de integración son:

- Manejo de argumentos y códigos de salida.
- Creación de .jnx para distintos JSON de prueba.
- Búsquedas con match exacto y parcial.
- Caso sin coincidencias que arrojen la salida [] esperada.
- Formato de salida: arreglo JSON con llaves path y value.
- Advertencia por índice potencialmente desactualizado.

También es importante indicar que algunas pruebas unitarias validan explícitamente casos que van a ocasionar algún error para validar el comportamiento del programa, por ejemplo:

- Cargar un archivo .jnx con un header inválido y asegurar que falle de forma controlada.
- Probar un patrón regex inválido y asegurar que se reporte.
- Forzar un buffer insuficiente en extracción y validar que el error se reporta sin corromper salida.

Análisis de resultados

Correctitud funcional

La ejecución de pruebas unitarias indica que:

- El parser construye paths coherentes y offsets consistentes para objetos, arreglos, strings, números, booleanos y null.
- El formato .jnx se escribe y se puede volver a cargar conservando exactamente el path y los offsets.
- El motor de regex maneja patrones válidos, sin match (retornando resultado vacío pero no NULL), y patrones inválidos, fallando de forma controlada.
- El extractor retorna exactamente el fragmento de bytes esperado; y reporta correctamente errores de buffer insuficiente.

Además, la ejecución de las pruebas de integración indica que el sistema completo se comporta como se espera desde el punto de vista de usuario:

- Los subcomandos del CLI se comportan como deben y en los flujos planeados.
- El índice realmente se usa como "puente" entre la búsqueda por paths y la extracción por offsets.
- La salida mantiene estructura JSON válida y consistente en casos con y sin resultados.

Rendimiento esperado

- Indexación: Es un recorrido completo del archivo JSON, por lo que su costo es proporcional al tamaño del documento.
- Búsqueda: Carga el índice en memoria y evalúa la regex contra todos los paths, por lo que su costo es proporcional a la cantidad de entradas.
- Extracción: Cada match se resuelve leyendo un rango de bytes específico del JSON original, evitando parsear el documento completo.

Un punto clave es que la extracción por offsets evita volver a tokenizar o reconstruir el árbol JSON, se recupera el valor tal cual aparece en el archivo original. Esto tiene dos implicaciones: rapidez, ya que la lectura de rangos se hace directa, y fidelidad a los datos, ya que estos se representan según cómo estén en el archivo.

Observaciones y limitaciones

- Límites de path y profundidad: El path está acotado por una variable global (512), además, el parser maneja una profundidad máxima. Archivos JSON muy profundos o con keys extremadamente largas pueden truncar paths.
- Warning de truncamiento: El warning de compilación sugiere que, en casos extremos, el path puede truncarse en snprintf.
- Índice desactualizado: El CLI detecta con ayuda de timestamps si el .json es más nuevo que el .jnx y emite una advertencia. Esto ayuda a evitar resultados inconsistentes al modificar el JSON sin reindexar.

Conclusiones

La solución implementa correctamente una estrategia de indexación por paths + offsets que permite recuperar fragmentos de JSON de forma eficiente sin tener que hacer un parseo completo en cada consulta.

Las pruebas unitarias y de integración fueron ejecutadas exitosamente y respaldan la robustez del comportamiento central: creación de índice, búsqueda por regex y extracción exacta por offsets.

Como puntos de mejora razonables, se podría reforzar el manejo de paths largos (evitar truncamientos) y reforzar otras áreas del proyecto, sin embargo, en su estado actual, el proyecto cumple el objetivo: indexar y consultar porciones de JSON de forma rápida y reproducible.